UART_stopwatch_watch

2026.02.10

김수빈

목차

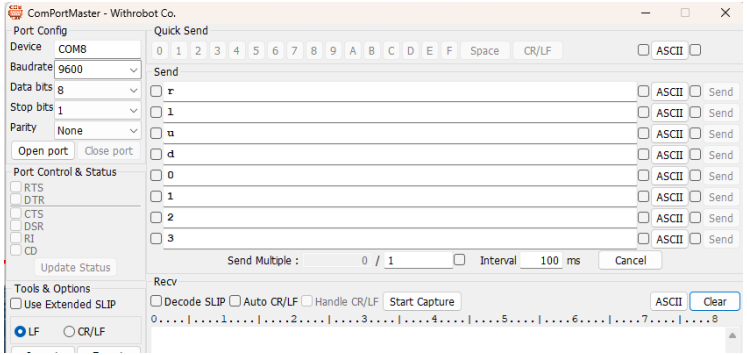
개요

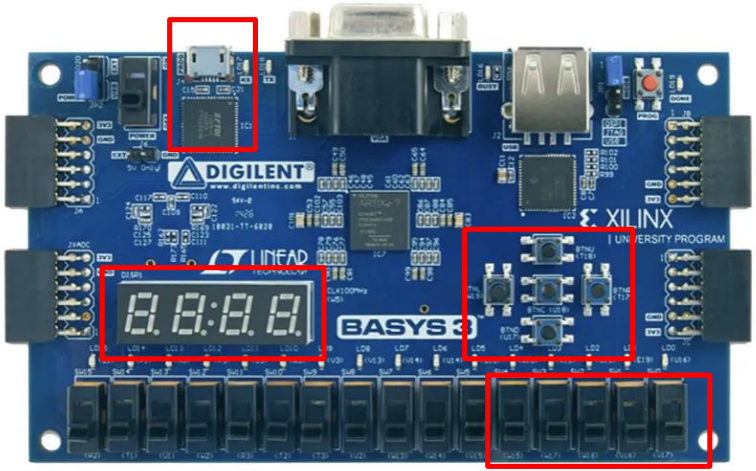
- **목표**
- UART 통신을 이용한 Stopwatch & Watch 기능 제어 시스템 설계

- **설계 내용**
- UART TX/RX FSM 및 ASM 설계
- UART는 비동기 통신이므로, 비트 시간 기준의 baud tick을 사용하여 구현
- ASCII Decoder를 통한 제어 신호 변환

- **검증 내용**
- Baud tick 동작 Simulation 검증
- UART 송수신 및 제어 기능 전체 Simulation 검증

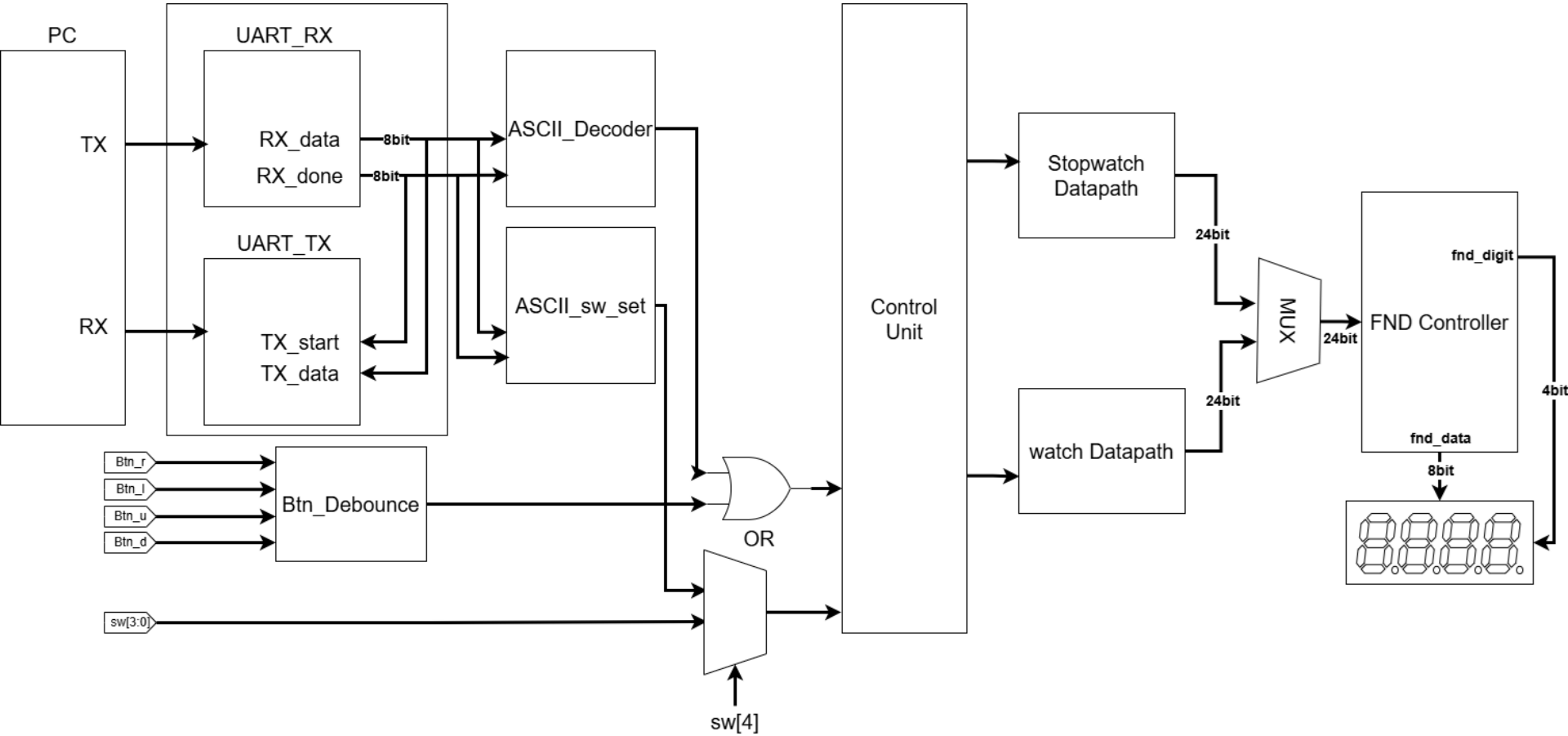
기능 설명



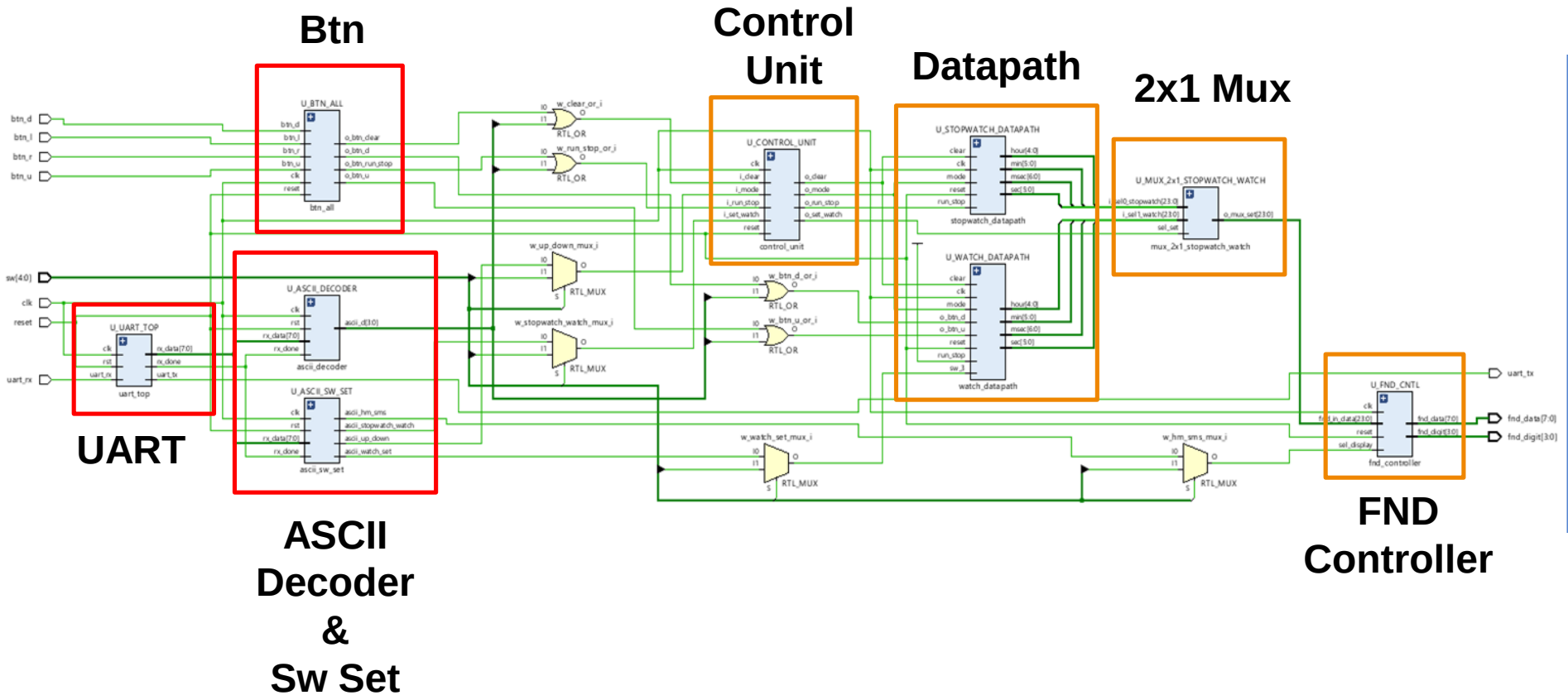


기능	btn / sw	uart
Run & Stop	btn r	r
Clear	btn l	l
Up mode / Down mode	sw[0]	0
Stopwatch / Watch	sw[1]	1
hour.min / sec.msec	sw[2]	2
Watch 시간 설정	sw[3] + btn u & btn d	3 + u & d
uart / sw	sw[4]	

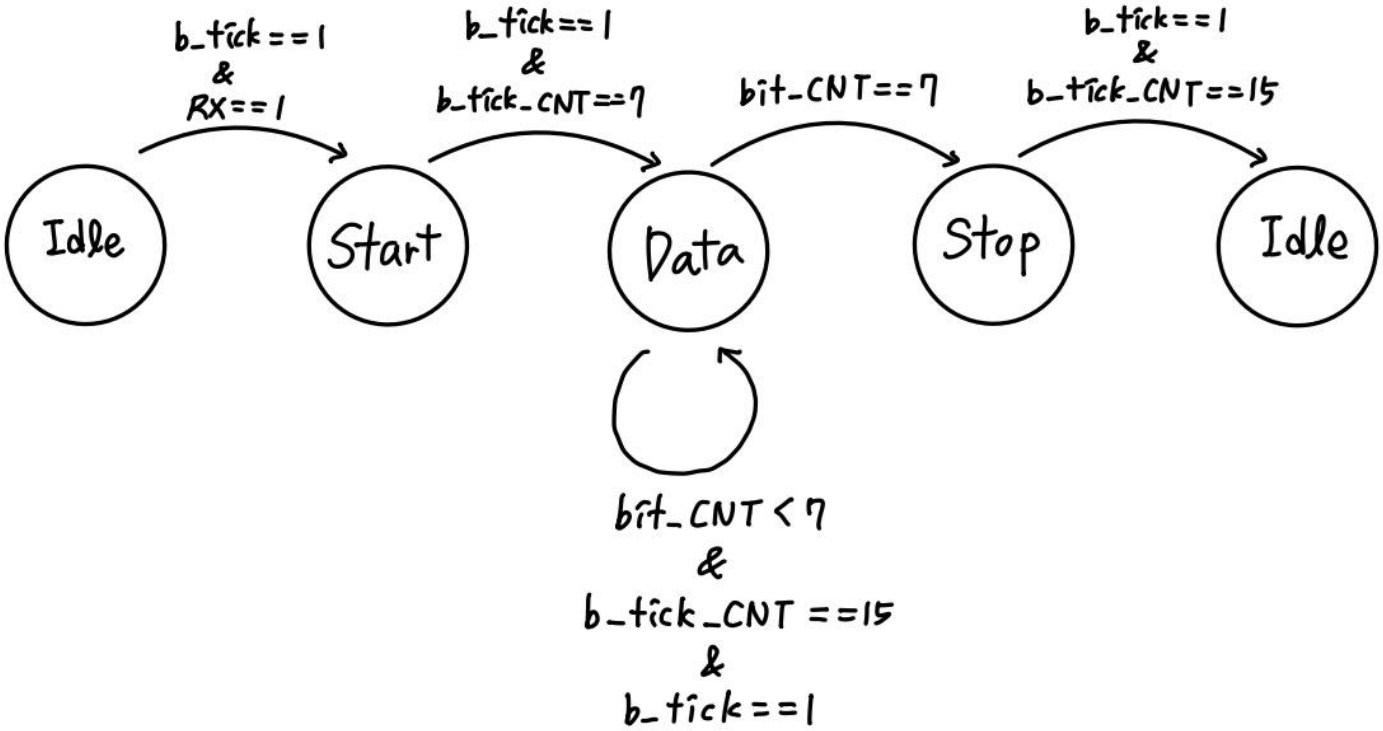
Block diagram



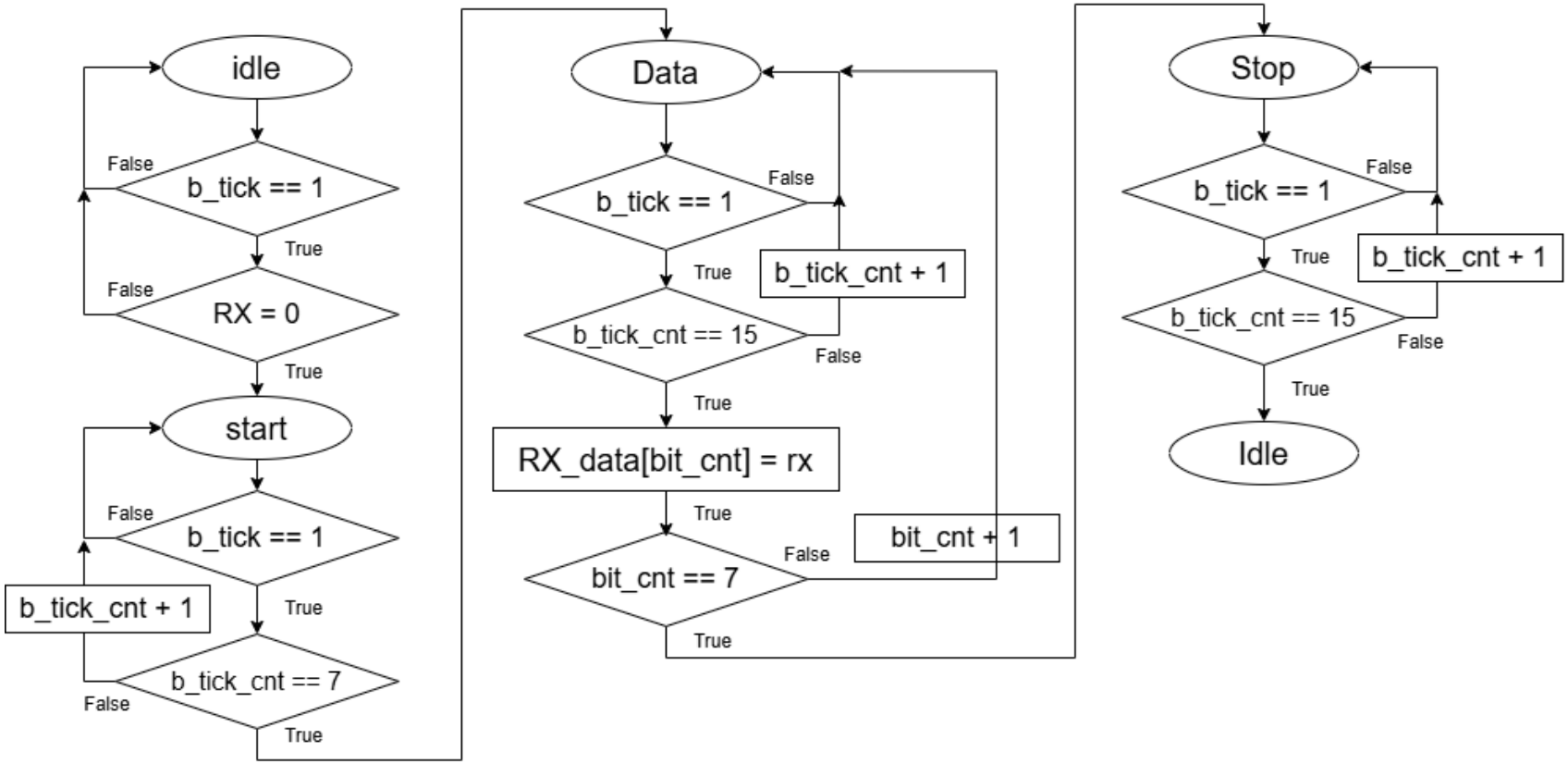
Schematic


Btn
Control Unit
Datapath
2x1 Mux
UART
ASCII Decoder & Sw Set
FND Controller
U_BTN_ALL
btn_all
U_CONTROL_UNIT
control_unit
U_STOPWATCH_DATAPATH
stopwatch_datapath
U_WATCH_DATAPATH
watch_datapath
U_MUX_2x1_STOPWATCH_WATCH
mux_2x1_stopwatch_watch
U_UART_TOP
uart_top
U_ASCII_DECODER
ascii_decoder
U_ASCII_SW_SET
ascii_sw_set
U_FND_CNTL
fnd_controller
RTL_OR
RTL_MUX
btn_d
btn_l
btn_r
btn_u
sw[4:0]
clk
reset
uart_rx
uart_tx
fnd_data[7:0]
fnd_digit[3:0]

RX_FSM


b_tick == 1
&
RX == 1
b_tick == 1
&
b_tick_CNT == 7
bit_CNT == 7
b_tick == 1
&
b_tick_CNT == 15
Idle
Start
Data
Stop
Idle
bit_CNT < 7
&
b_tick_CNT == 15
&
b_tick == 1

RX_ASM


idle
b_tick == 1
False
True
RX = 0
False
True
start
b_tick == 1
False
True
b_tick_cnt + 1
b_tick_cnt == 7
False
True
Data
b_tick == 1
False
True
b_tick_cnt + 1
b_tick_cnt == 15
False
True
RX_data[bit_cnt] = rx
True
bit_cnt + 1
bit_cnt == 7
False
True
Stop
b_tick == 1
False
True
b_tick_cnt + 1
b_tick_cnt == 15
False
True
Idle

Code_UART_RX

```
always @(*) begin
    n_state         = c_state;
    b_tick_cnt_next = b_tick_cnt_reg;
    bit_cnt_next    = bit_cnt_reg;
    done_next       = done_reg;
    buf_next        = buf_reg;

    case (c_state)
        IDLE: begin
            b_tick_cnt_next = 5'd0;
            bit_cnt_next    = 3'd0;
            done_next       = 1'b0;

            if (b_tick & !rx) begin
                buf_next = 8'd0;
                n_state  = START;
            end
        end
        START: begin
            if (b_tick) begin
                if (b_tick_cnt_reg == 7) begin
                    b_tick_cnt_next = 0;
                    n_state = DATA;
                end else begin
                    b_tick_cnt_next = b_tick_cnt_reg + 1;
```

IDLE

START

```
        DATA: begin
            if (b_tick) begin
                if (b_tick_cnt_reg == 4'd15) begin
                    b_tick_cnt_next = 5'd0;
                    buf_next = {rx, buf_reg[7:1]};
                    if (bit_cnt_reg == 4'd7) begin
                        n_state = STOP;
                    end else begin
                        bit_cnt_next = bit_cnt_reg + 1;
                    end
                end else begin
                    b_tick_cnt_next = b_tick_cnt_reg + 1;
                end
            end
        end
```

DATA

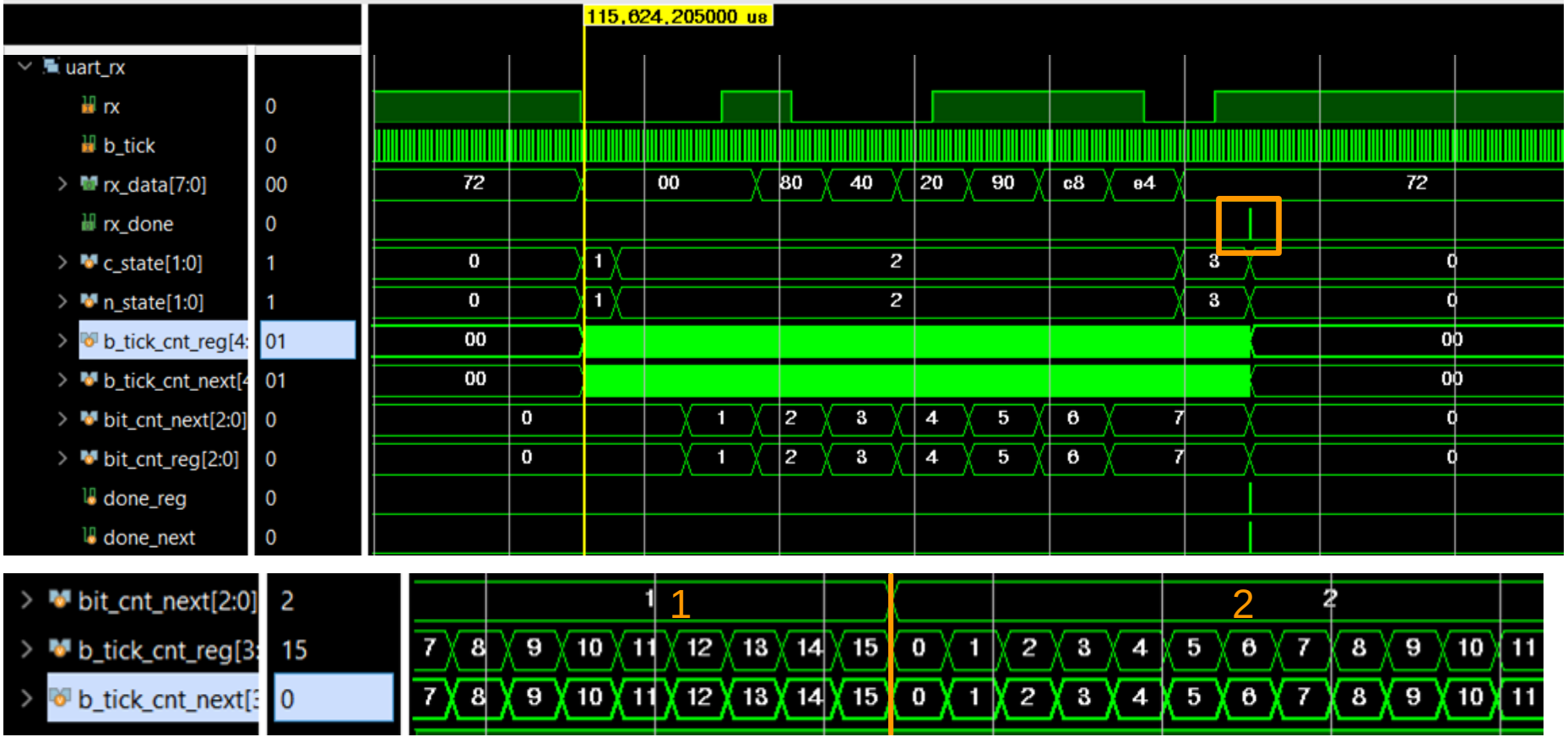
수신 shift

```
        STOP: begin
            if (b_tick) begin
                if (b_tick_cnt_reg == 15) begin
                    n_state    = IDLE;
                    done_next = 1'b1;
                end else begin
                    b_tick_cnt_next = b_tick_cnt_reg + 1;
```

STOP

{1’b0, data_in_buf_reg[7:1]} : d7 d6 d5 d4 d3 d2 d1 d0 → **rx** b7 b6 b5 b4 b3 b2 b1

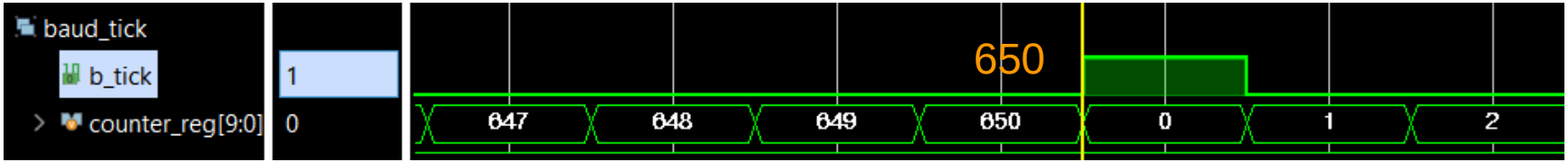
Simulation_UART_RX



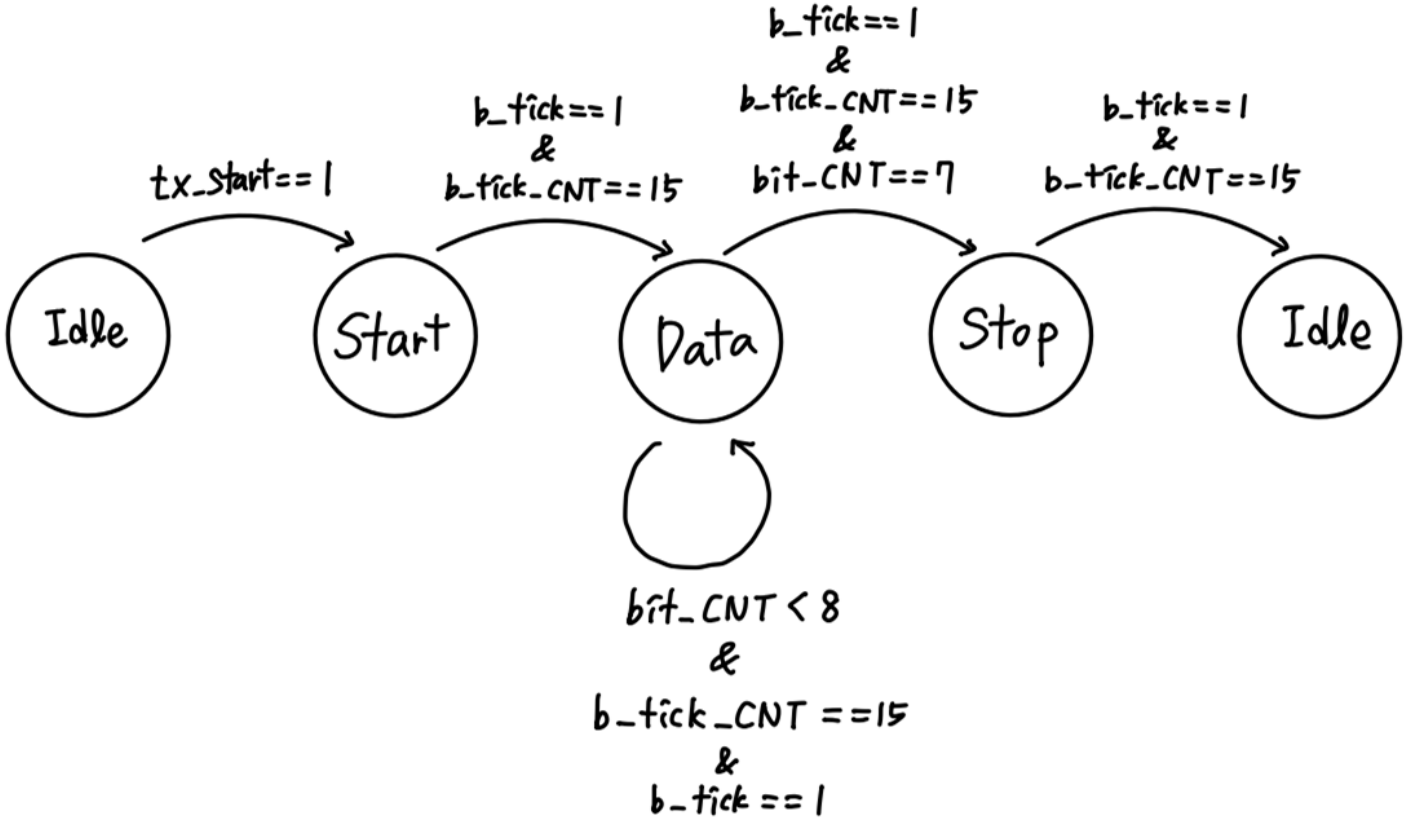
Baud_tick

```
parameter BAUDRATE = 9600 * 16;
parameter F_COUNT = 100_000_000 / BAUDRATE;

reg [$clog2(F_COUNT) - 1:0] counter_reg;

always @(posedge clk, posedge rst) begin
    if (rst) begin
        counter_reg <= 0;
        b_tick <= 1'b0;
    end else begin
        counter_reg <= counter_reg + 1;
        if (counter_reg == (F_COUNT - 1)) begin
            counter_reg <= 0;
            b_tick <= 1'b1;
        end else begin
            b_tick <= 1'b0;
        end
    end
```

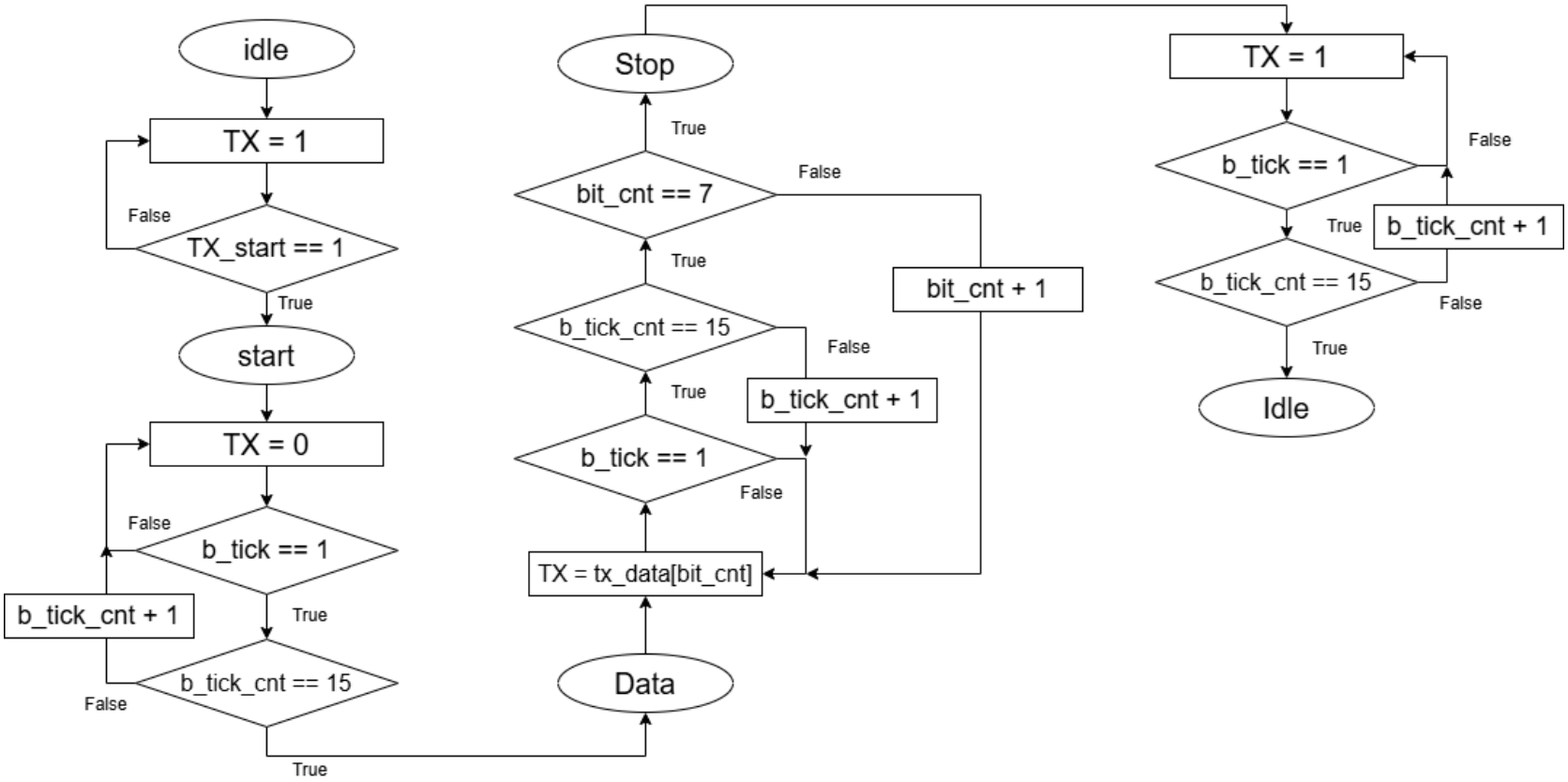
Baud_tick은
6.51us 마다
한 번 씩 발생



TX_FSM


Idle
Start
Data
Stop
Idle
tx_start==1
b_tick==1
&
b_tick_CNT==15
b_tick==1
&
b_tick_CNT==15
&
bit_CNT==7
b_tick==1
&
b_tick_CNT==15
bit_CNT<8
&
b_tick_CNT==15
&
b_tick==1

TX_ASM


idle
TX = 1
False
TX_start == 1
True
start
TX = 0
False
b_tick == 1
True
b_tick_cnt + 1
False
b_tick_cnt == 15
True
Data
TX = tx_data[bit_cnt]
b_tick == 1
False
True
b_tick_cnt + 1
b_tick_cnt == 15
False
True
bit_cnt + 1
bit_cnt == 7
False
True
Stop
TX = 1
b_tick == 1
False
True
b_tick_cnt + 1
b_tick_cnt == 15
False
True
Idle

Code_UART_TX

```
case (c_state)
    IDLE: begin
        tx_next         = 1'b1;
        bit_cnt_next    = 1'b0;
        b_tick_cnt_next = 4'h0;
        busy_next       = 1'b0;
        done_next       = 1'b0;
        if (tx_start == 1) begin
            n_state          = START;
            busy_next        = 1'b1;
            //start 인지했을 때 넣기
            data_in_buf_next = tx_data;
        end
    end
```

IDLE

```
    START: begin
        //to start uart frame start bit
        tx_next = 1'b0;
        if (b_tick == 1) begin
            if (b_tick_cnt_reg == 15) begin
                n_state = DATA;
                b_tick_cnt_next = 4'h0;
            end else begin
                b_tick_cnt_next = b_tick_cnt_reg + 1;
            end
```

START

```
    DATA: begin
        tx_next = data_in_buf_reg[0];
        if (b_tick == 1) begin
            if (b_tick_cnt_reg == 15) begin
                if (bit_cnt_reg == 7) begin
                    b_tick_cnt_next = 4'h0;
                    n_state = STOP;
                end else begin
                    b_tick_cnt_next = 4'h0;
                    bit_cnt_next = bit_cnt_reg + 1;
                    data_in_buf_next = {1'b0, data_in_buf_reg[7:1]};
                    n_state = DATA;
                end
            end else begin
                b_tick_cnt_next = b_tick_cnt_reg + 1;
            end
        end
    end
```

DATA

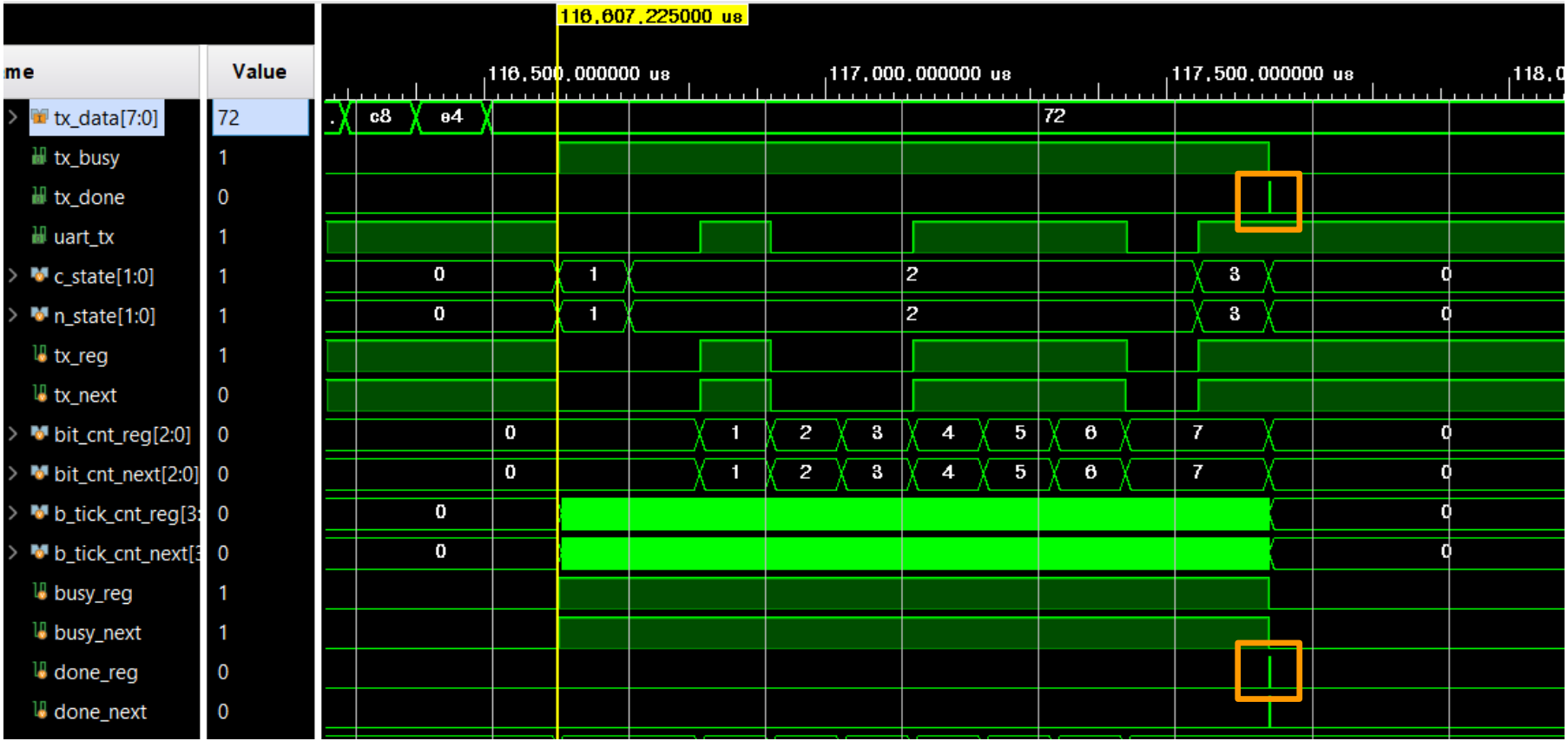
송신 shift

```
    STOP: begin
        tx_next = 1'b1;
        if (b_tick == 1) begin
            if (b_tick_cnt_reg == 15) begin
                done_next = 1'b1;
                n_state   = IDLE;
            end else begin
                b_tick_cnt_next = b_tick_cnt_reg + 1;
            end
```

STOP

{1’b0, data_in_buf_reg[7:1]} : d7 d6 d5 d4 d3 d2 d1 d0 → **0** d7 d6 d5 d4 d3 d2 d1

Simulation_UART_TX



Code_ASCII Decoder

```
always @(posedge clk, posedge rst) begin
    if (rst) begin
        ascii_d <= 4'b0000;
    end else begin
        ascii_d <= 4'b0000;
        if (rx_done) begin
            case (rx_data)
                8'h72: ascii_d <= 4'b0001; //r
                8'h6C: ascii_d <= 4'b0010; //l
                8'h75: ascii_d <= 4'b0100; //u
                8'h64: ascii_d <= 4'b1000; //d
            endcase
```

- 매 CLK마다 기본값으로 초기화
- 한 번에 하나의 제어 비트만 활성화

```
assign w_run_stop_or        = o_btn_run_stop | w_ascii_d[0];
assign w_clear_or           = o_btn_clear | w_ascii_d[1];
assign w_btn_u_or           = o_btn_u | w_ascii_d[2];
assign w_btn_d_or           = o_btn_d | w_ascii_d[3];
```

기존 버튼과 OR 연산으로 선택

Code_ASCII_sw_set

```
always @(posedge clk, posedge rst) begin
    if (rst) begin
        ascii_up_down         <= 0;
        ascii_stopwatch_watch <= 0;
        ascii_hm_sms          <= 0;
        ascii_watch_set       <= 0;
    end else begin
        if (rx_done) begin
            case (rx_data)
                8'h30:   ascii_up_down <= ~ascii_up_down;
                8'h31:   ascii_stopwatch_watch <= ~ascii_stopwatch_watch;
                8'h32:   ascii_hm_sms <= ~ascii_hm_sms;
                8'h33:   ascii_watch_set <= ~ascii_watch_set;
                default: ;
            endcase
```

UART 값이 들어오면
값이 toggle 되도록 설정

```
assign w_uart_sw_sel = sw[4];

assign w_up_down_mux = w_uart_sw_sel ? w_ascii_up_down : sw[0];
assign w_stopwatch_watch_mux =  w_uart_sw_sel ? w_ascii_stopwatch_watch : sw[1];
assign w_hm_sms_mux =  w_uart_sw_sel ? w_ascii_hm_sms : sw[2];
assign w_watch_set_mux =  w_uart_sw_sel ? w_ascii_watch_set : sw[3];
```

sw[4]로 모드 변환
기존 sw와
3항 연산자로 선택

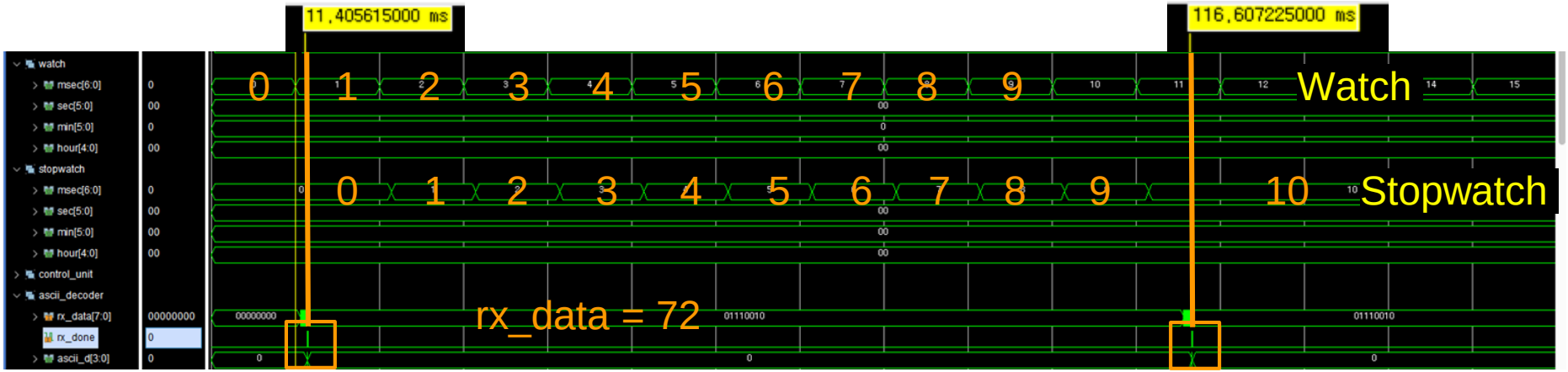
Code_ASCII Decoder & sw_set port 연결

```
control_unit U_CONTROL_UNIT (
    .clk        (clk),
    .reset      (reset),
    .i_mode     (w_up_down_mux),
    .i_run_stop (w_run_stop_or),
    .i_clear    (w_clear_or),
    .i_set_watch(w_stopwatch_watch_mux),
    .o_mode     (w_mode),
    .o_run_stop (w_run_stop),
    .o_clear    (w_clear),
    .o_set_watch(w_set_watch)
);
```

```
fnd_controller U_FND_CNTL (
    .clk        (clk),
    .reset      (reset),
    .sel_display(w_hm_sms_mux),
    .fnd_in_data(o_mux_set),
    .fnd_digit  (fnd_digit),
    .fnd_data   (fnd_data)
);
```

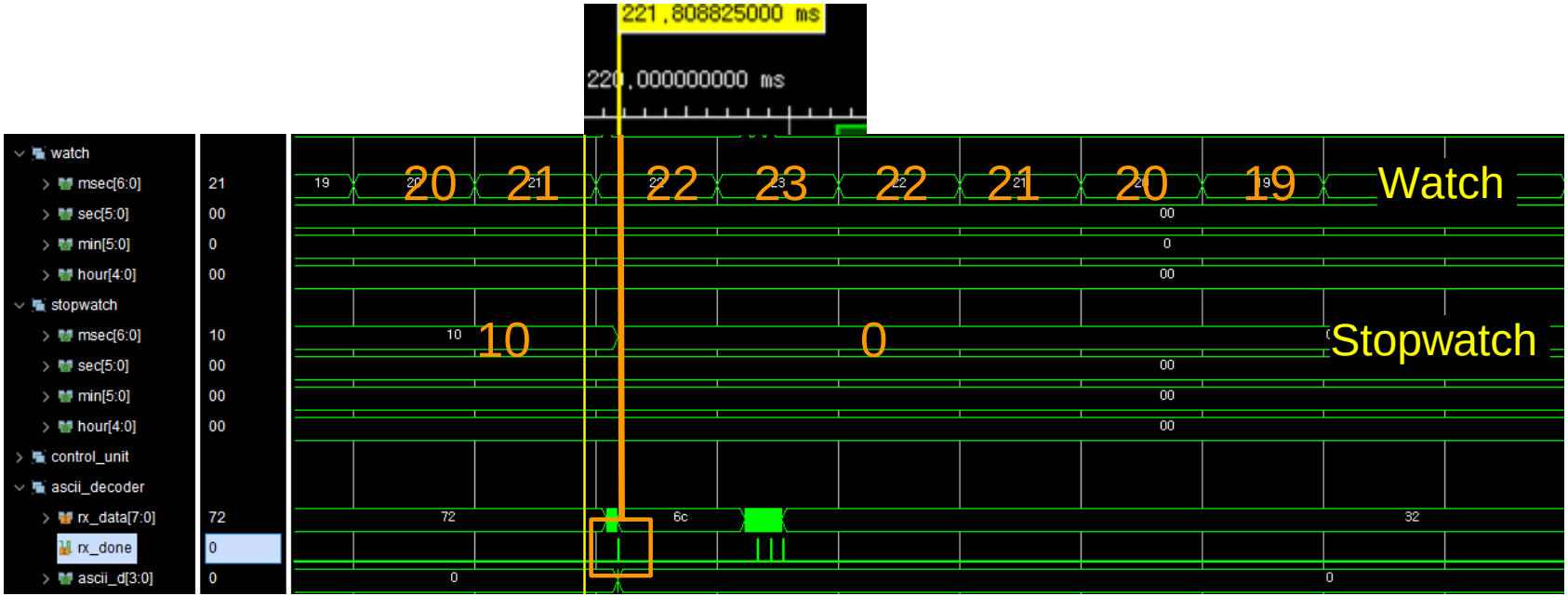
```
watch_datapath U_WATCH_DATAPATH (
    .clk(clk),
    .reset(reset),
    .mode(w_mode),
    .clear(w_clear),
    .run_stop(1'b1),
    .sw_3(w_watch_set_mux),
    .o_btn_u(w_btn_u_or),
    .o_btn_d(w_btn_d_or),
    .msec(w_watch_time[6:0]),
    .sec(w_watch_time[12:7]),
    .min(w_watch_time[18:13]),
    .hour(w_watch_time[23:19])
);
```

Simulation_count Run & Stop



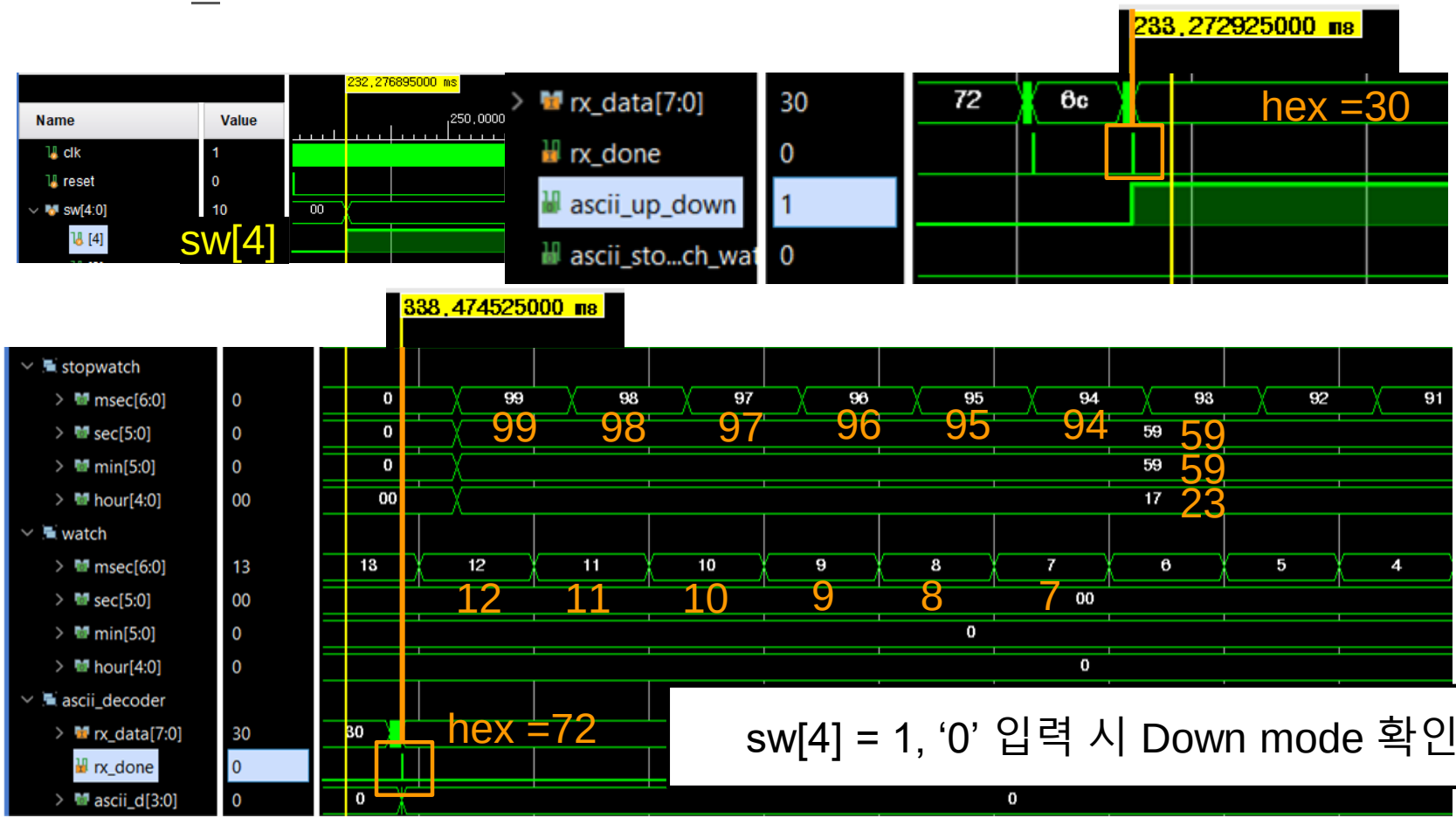
‘r’ 입력 시, stopwatch 증가하는 것과 멈추는 것 확인

Simulation_stopwatch count Clear

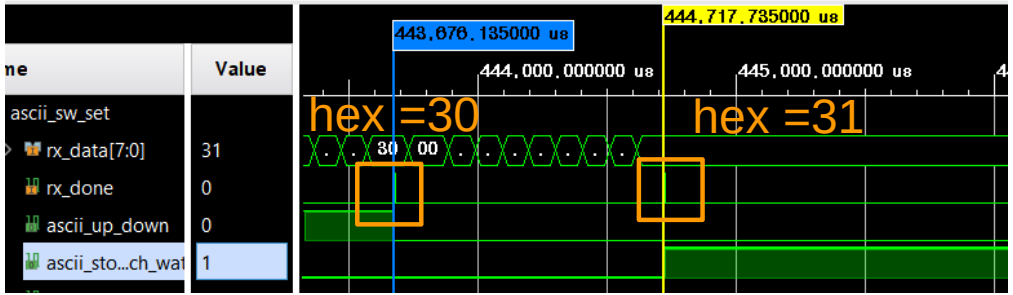


‘l’ 입력 시, stopwatch 초기화 확인

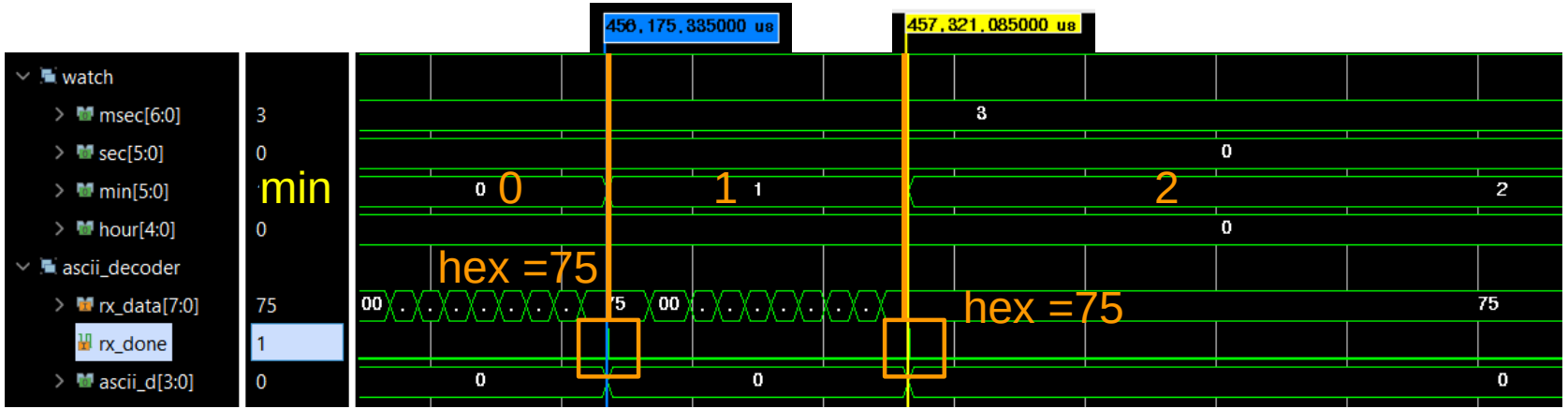
Simulation_Down mode



Simulation_시간 설정_min up

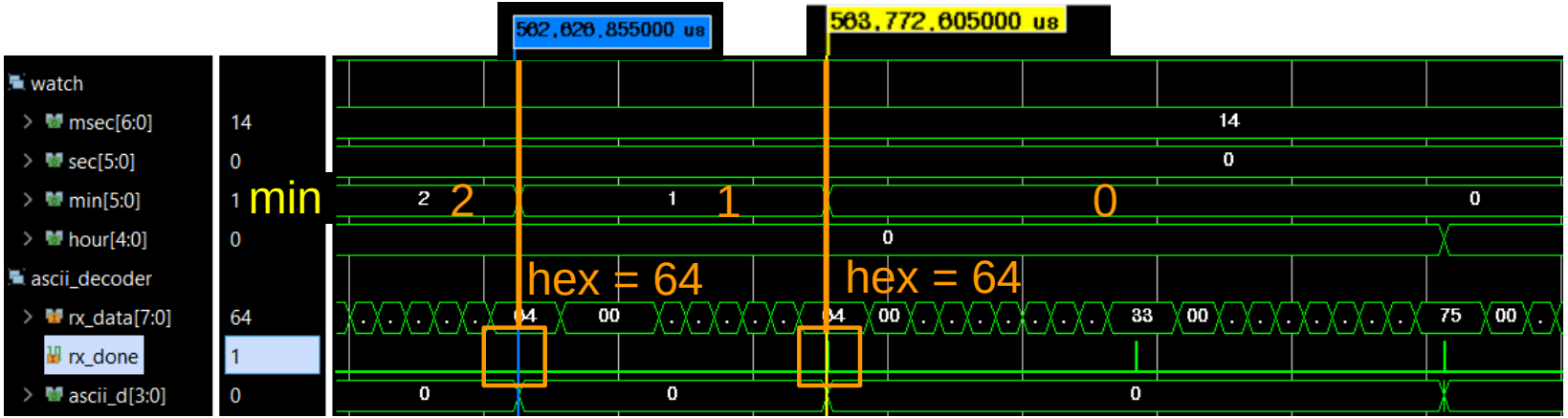


Run 모드, '1' 입력으로
Watch 모드 전환



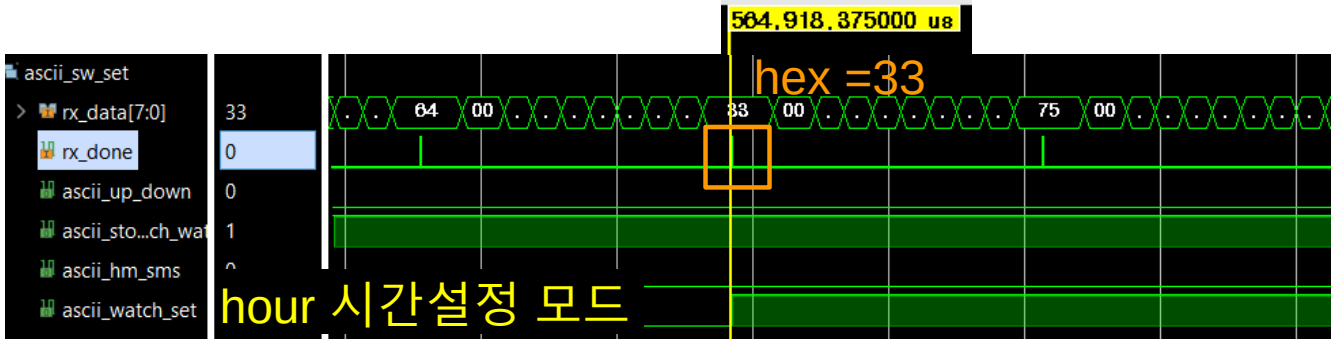
'u' 입력 시, min 값 증가 확인

Simulation_시간 설정_min down

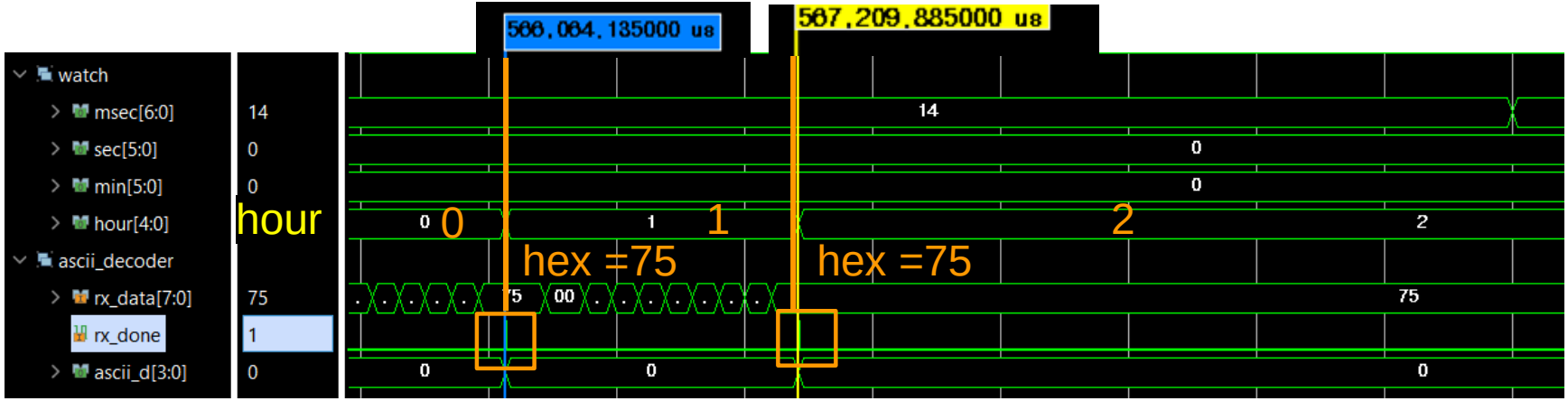


‘d’ 입력 시, min 값 감소 확인

Simulation_시간 설정_hour up

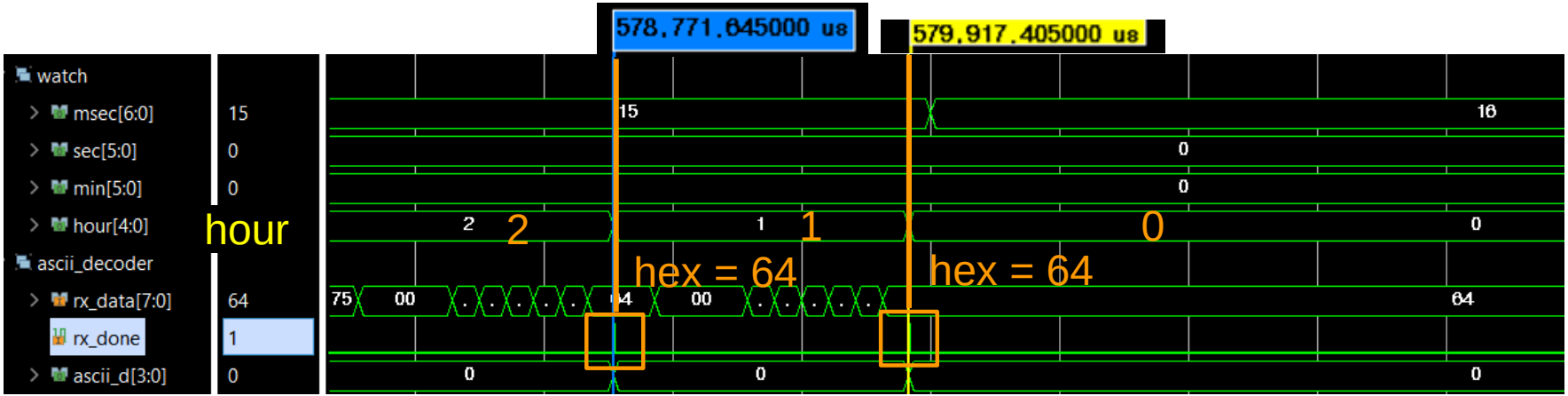


'3' 입력으로
hour 설정
모드 전환



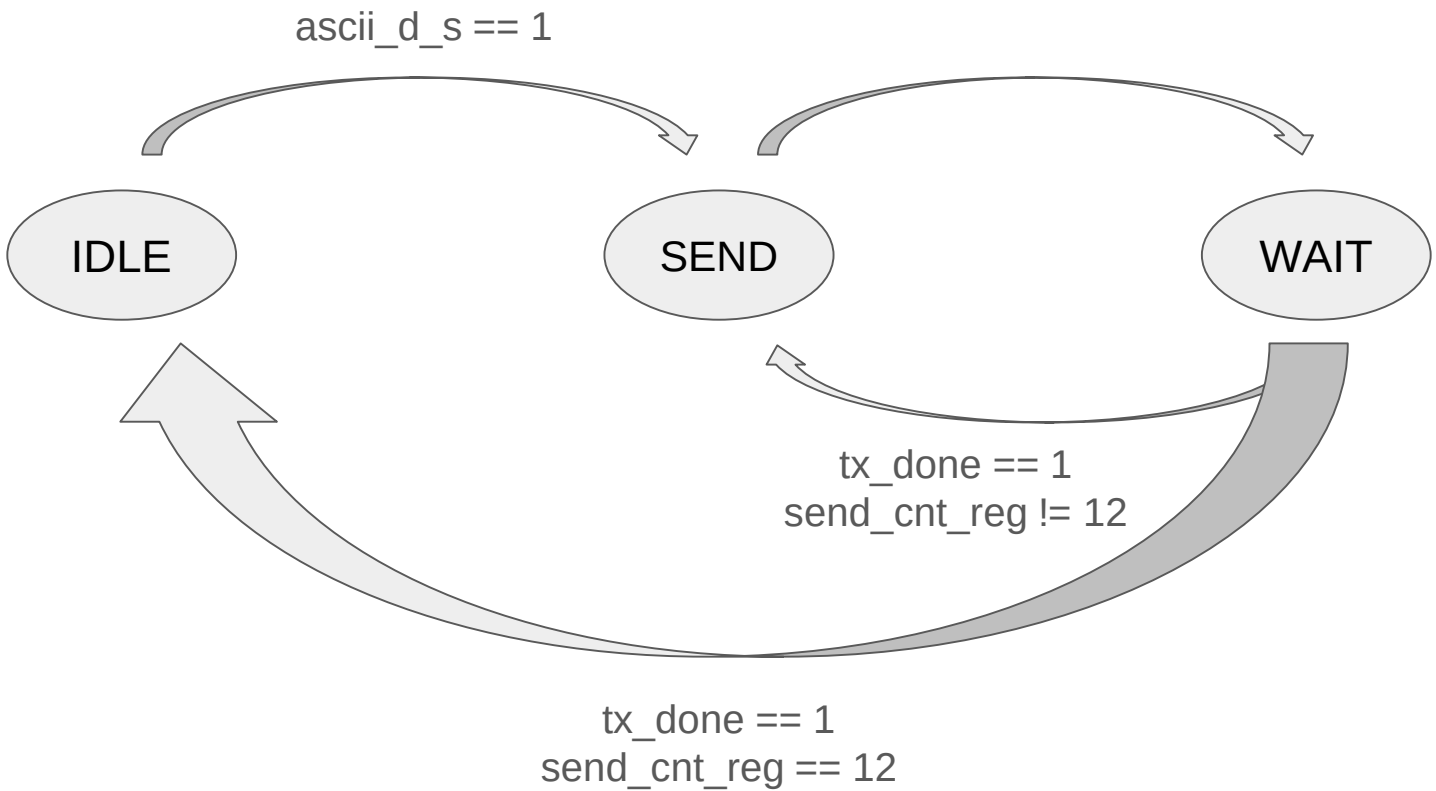
'u' 입력 시, hour 값 증가 확인

Simulation_시간 설정_hour down

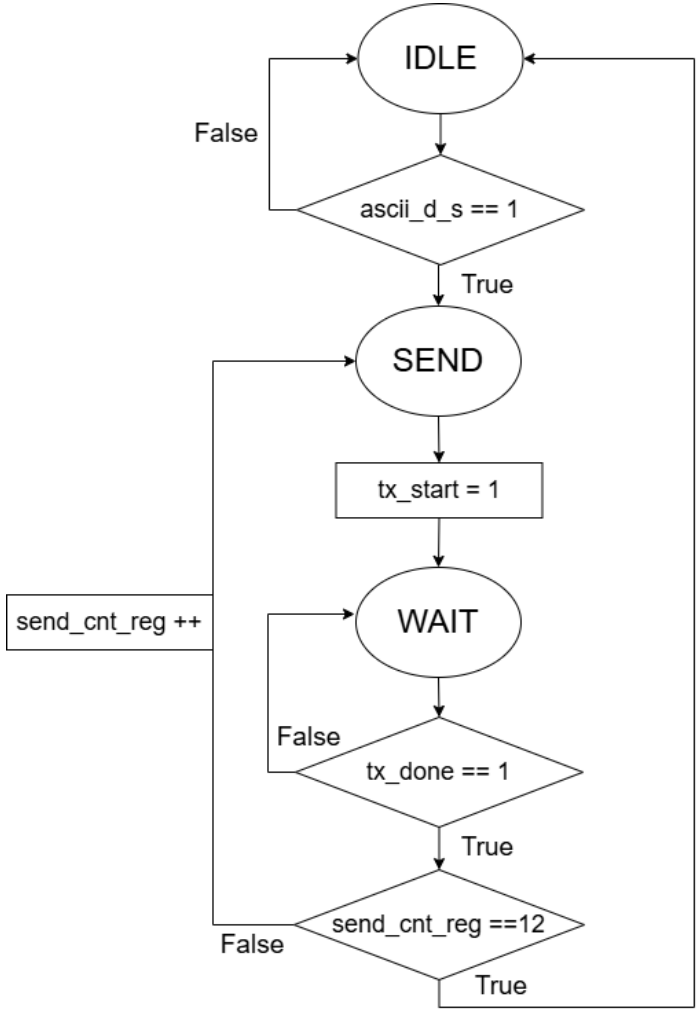


‘d’ 입력 시, hour 값 감소 확인

Sender_FSM


ascii_d_s == 1
IDLE
SEND
WAIT
tx_done == 1
send_cnt_reg != 12
tx_done == 1
send_cnt_reg == 12

Sender_ASM


IDLE
False
ascii_d_s == 1
True
SEND
tx_start = 1
WAIT
send_cnt_reg ++
False
tx_done == 1
True
send_cnt_reg ==12
False
True

Code_Sender

IDLE

```
case (c_state)
    IDLE: begin
        if (ascii_d_s) begin
            n_state = SEND;
        end
    end
```

SEND

```
SEND: begin
    case (send_cnt_reg)
        4'd0: begin
            tx_data = ascii_hour_10;
            tx_start = 1;
            n_state = WAIT;
        end
        4'd1: begin
            tx_data = ascii_hour_1;
            tx_start = 1;
            n_state = WAIT;
        end
        4'd2: begin
            tx_data = 8'h3A;
            tx_start = 1;
            n_state = WAIT;
        end
        4'd3: begin
            tx_data = ascii_min_10;
            tx_start = 1;
            n_state = WAIT;
        end
        4'd4: begin
            tx_data = ascii_min_1;
            tx_start = 1;
            n_state = WAIT;
        end
        4'd5: begin
            tx_data = 8'h3A;
            tx_start = 1;
            n_state = WAIT;
        end
```

```
        4'd6: begin
            tx_data = ascii_sec_10;
            tx_start = 1;
            n_state = WAIT;
        end
        4'd7: begin
            tx_data = ascii_sec_1;
            tx_start = 1;
            n_state = WAIT;
        end
        4'd8: begin
            tx_data = 8'h3A;
            tx_start = 1;
            n_state = WAIT;
        end
        4'd9: begin
            tx_data = ascii_msec_10;
            tx_start = 1;
            n_state = WAIT;
        end
        4'd10: begin
            tx_data = ascii_msec_1;
            tx_start = 1;
            n_state = WAIT;
        end
        4'd11: begin
            tx_data = 8'h0A;
            tx_start = 1;
            n_state = WAIT;
        end
```

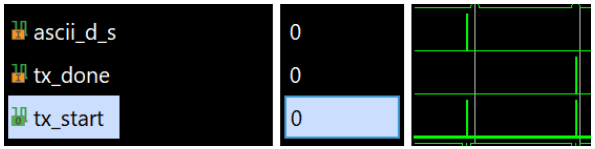
WAIT

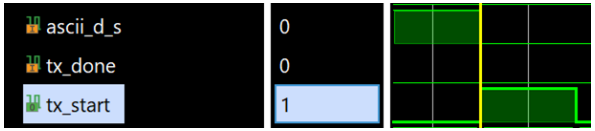
```
WAIT: begin
    if (tx_done) begin
        if (send_cnt_reg == 12) begin
            send_cnt_next = 0;
            n_state = IDLE;
        end else begin
            send_cnt_next = send_cnt_reg + 1;
            n_state = SEND;
        end
    end

end
```

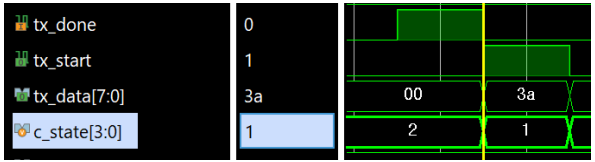
BCD

```
always @(bcd_sender) begin
    case (bcd_sender)
        4'd0: send_data = 8'h30;
        4'd1: send_data = 8'h31;
        4'd2: send_data = 8'h32;
        4'd3: send_data = 8'h33;
        4'd4: send_data = 8'h34;
        4'd5: send_data = 8'h35;
        4'd6: send_data = 8'h36;
        4'd7: send_data = 8'h37;
        4'd8: send_data = 8'h38;
        4'd9: send_data = 8'h39;
        default: send_data = 8'h20;
    endcase
end
```

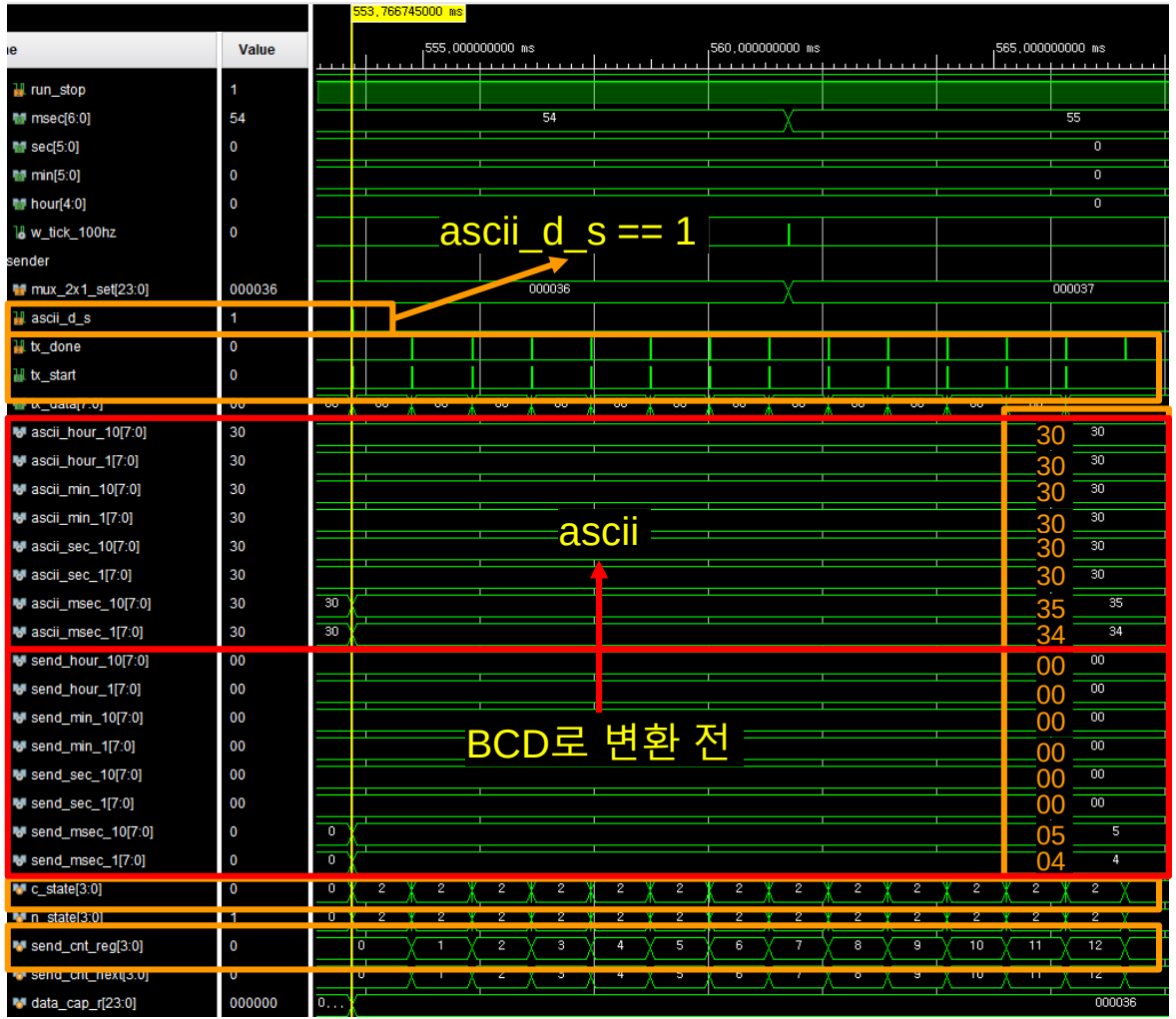
Simulation_Stopwatch





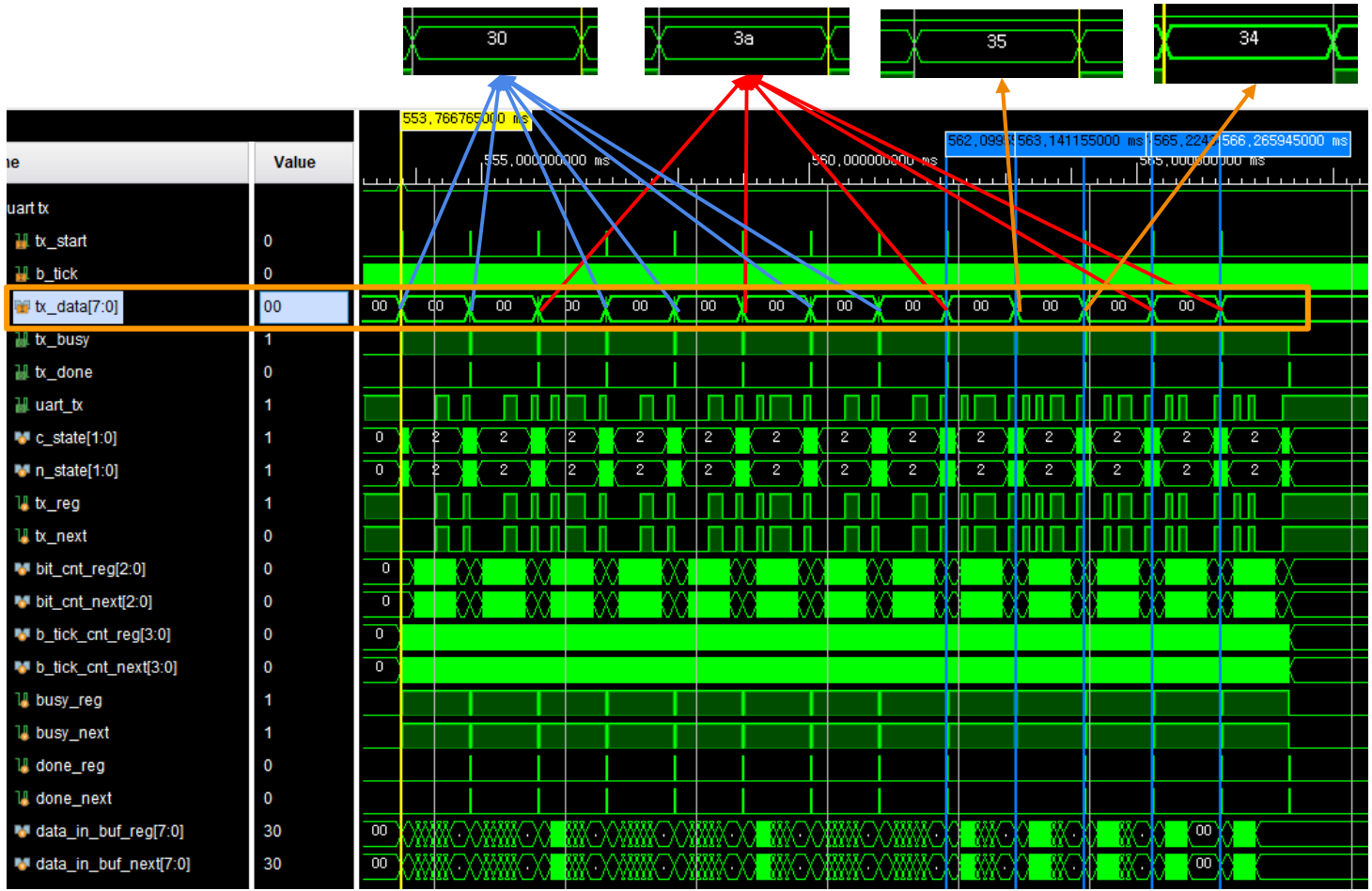


ascii_d_s 신호 뜨고, tx_start 신호 뜸
tx_done이 뜨고 state 1인 SEND 상태로 가면서 다시 tx_start 신호 뜸
send_cnt_reg == 12까지 반복



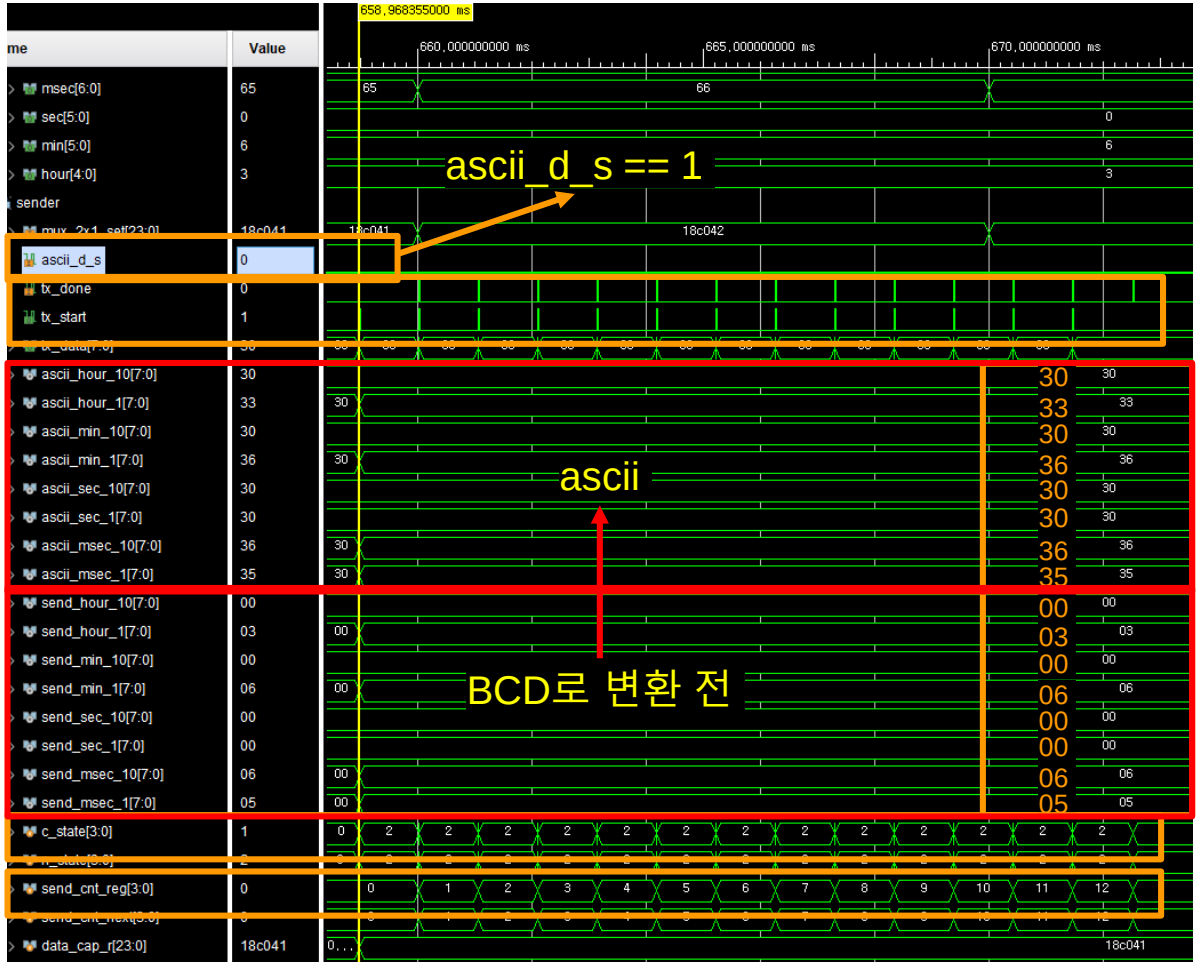
Simulation

00:00:00:54



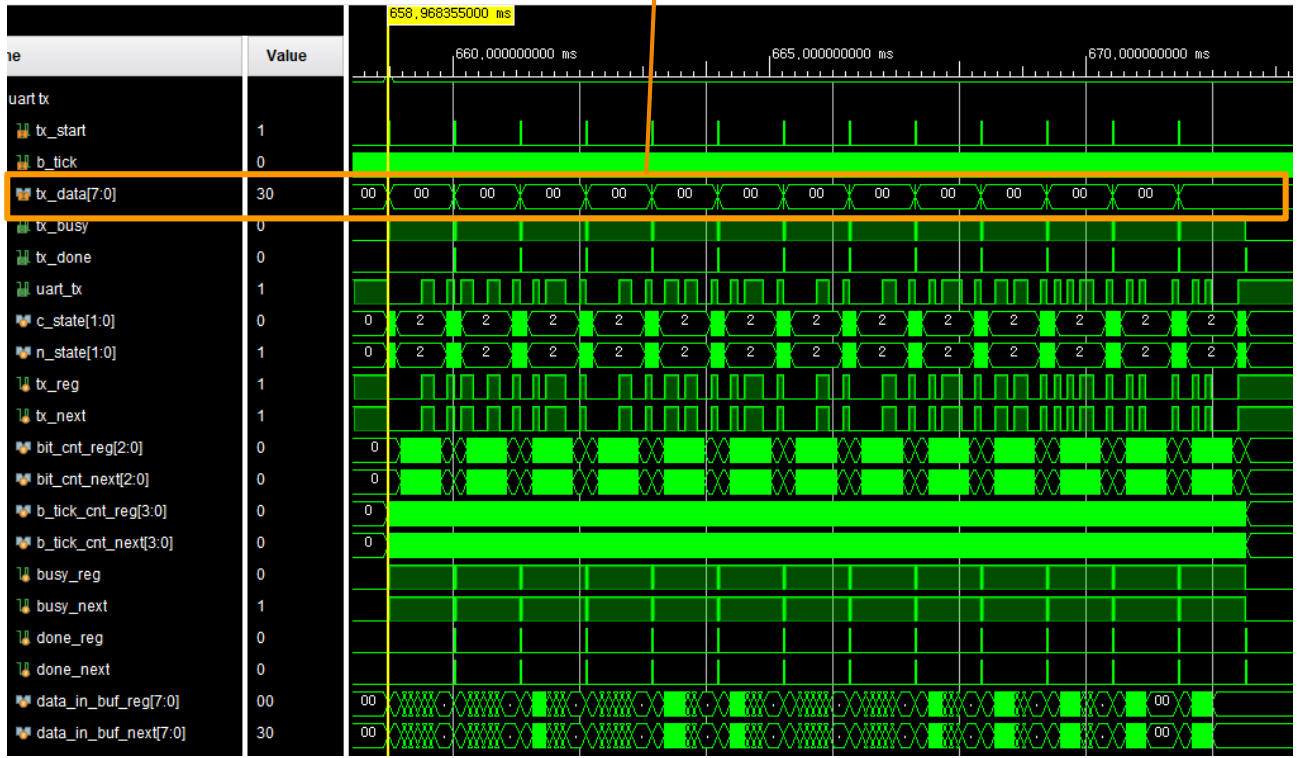
Simulation_Watch

ascii_d_s 신호 뜨고, tx_start 신호 뜸
tx_done이 뜨고 state 1인 SEND 상태로 가면서 다시 tx_start 신호 뜸
send_cnt_reg == 12까지 반복

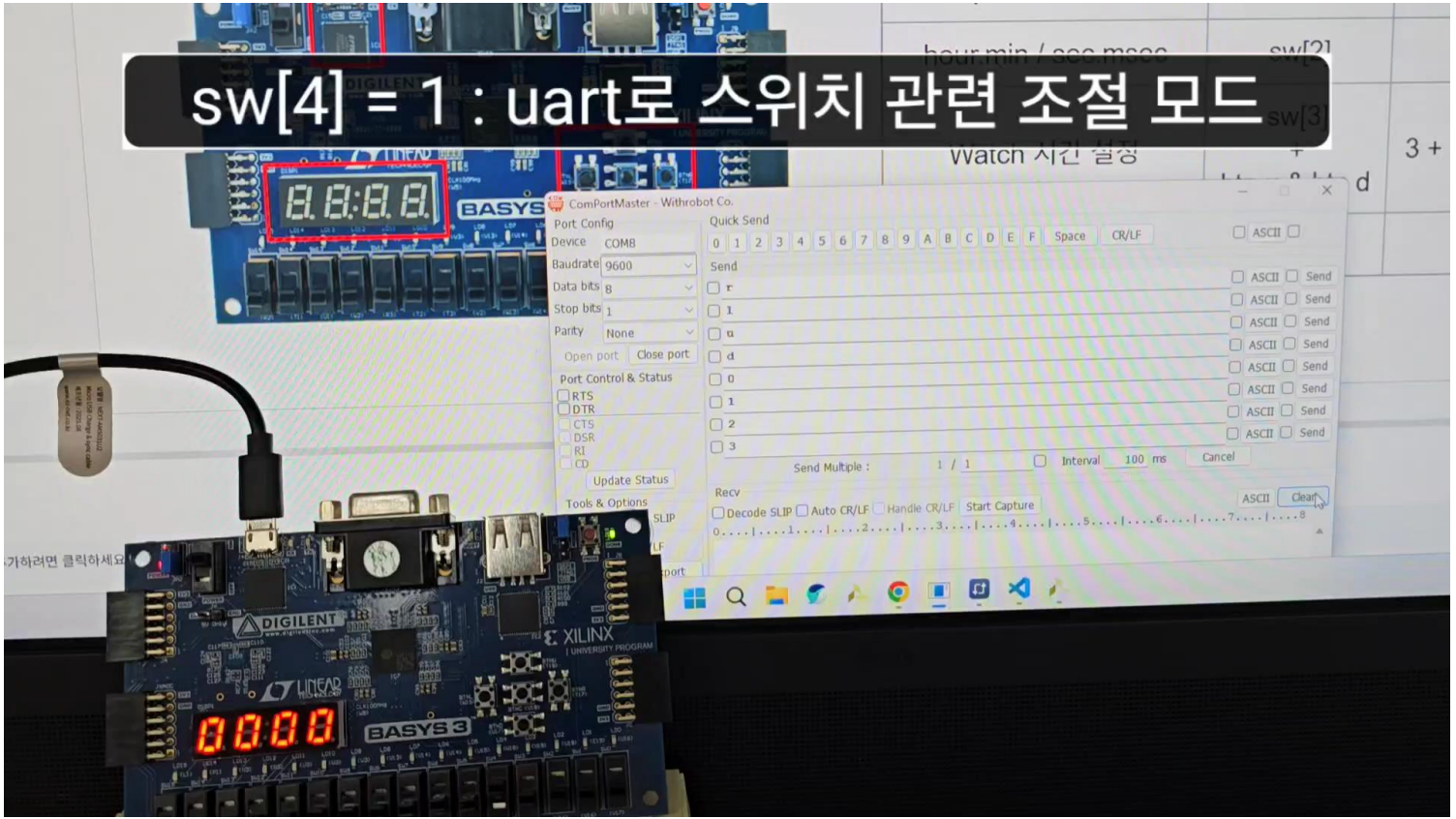


Simulation

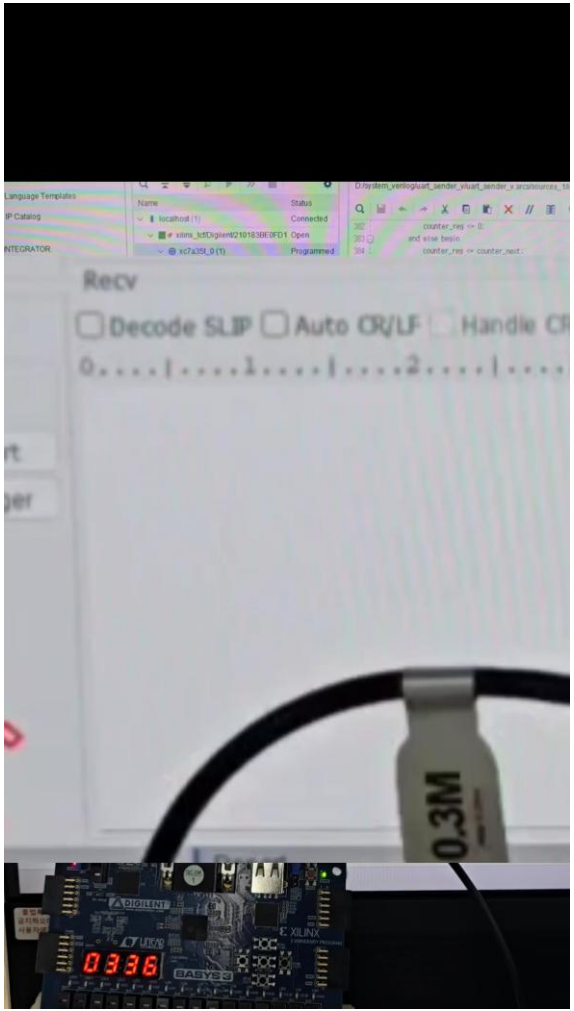
03:06:00:65



동작 영상


sw[4] = 1 : uart로 스위치 관련 조절 모드
ComPortMaster - Withrobot Co.
Port Config
Device COM8
Baudrate 9600
Data bits 8
Stop bits 1
Parity None
Open port
Close port
Port Control & Status
RTS
DTR
CTS
DSR
RI
CD
Update Status
Tools & Options
Quick Send
Space
CR/LF
ASCII
Send
Send Multiple :
Interval 100 ms
Cancel
Recv
Decode SLIP
Auto CR/LF
Handle CR/LF
Start Capture
Clear
DIGILENT
XILINX
UNIVERSITY PROGRAM
BASYS 3

동작영상_Sender



Trouble Shooting_sw와 uart OR 시 문제점

sw로 제어하던 것을 UART로 제어하는 것을 추가할 때

```
assign w_up_down_or          = w_ascii_up_down | sw[0];
assign w_stopwatch_watch_or = w_ascii_stopwatch_watch | sw[1];
assign w_hm_sms_or           = w_ascii_hm_sms | sw[2];
assign w_watch_set_or        = w_ascii_watch_set | sw[3];
```

OR 이용

↓

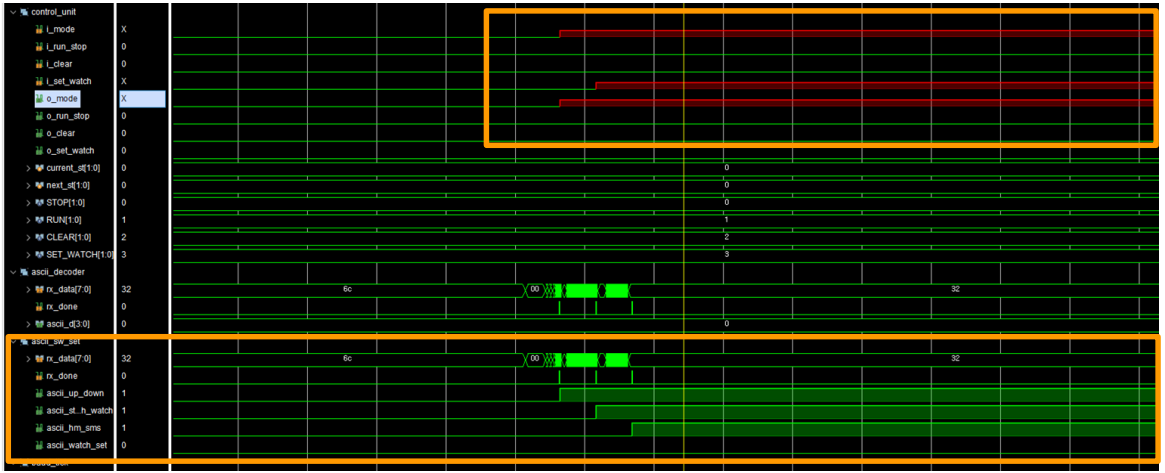
두 입력 중 하나가 1이면, 다른 입력이 동작하지 않는 문제

↓

```
assign w_uart_sw_sel = sw[4];

assign w_up_down_mux = w_uart_sw_sel ? w_ascii_up_down : sw[0];
assign w_stopwatch_watch_mux =  w_uart_sw_sel ? w_ascii_stopwatch_watch : sw[1];
assign w_hm_sms_mux =  w_uart_sw_sel ? w_ascii_hm_sms : sw[2];
assign w_watch_set_mux =  w_uart_sw_sel ? w_ascii_watch_set : sw[3];
```

3항 연산자 이용

Trouble Shooting_Simulation 시나리오 오류



sw[4] = 1

sw[4] 모드 설정 없이 UART 값 전송 시 Short 발생
→ Short

```
sw[4] = 1;
#1000;

test_data = 8'h30; // sw[0]
uart_sender();

test_data = 8'h31; // sw[1]
uart_sender();

test_data = 8'h32; // sw[2]
uart_sender();

for(j = 0;j < 1000;j = j + 1) begin
    #(BAUD_PERIOD);
end

test_data = 8'h72; //r
uart_sender();
```

감사합니다.